

Bm

Boyer–Moore 算法

前置知识：[前缀函数与 KMP 算法](#)。

KMP 算法将前缀匹配的信息用到了极致，

而 BM 算法背后的基本思想是通过后缀匹配获得比前缀匹配更多的信息来实现更快的字符跳转。

引入

想象一下，如果我们的的模式字符串 P ，被放在文本字符串 T 的左手起头部，使它们的第一个字符对齐。

在这里做定义，往后不赘述：

$|P|$ 的长度为 m ，特别地对于从 0 开始的串来说，规定 $P[m]$ 为串最后一个字符的位置；

$|T|$ 的长度为 n ，。

假如我们知道了 T 的第 i 个字符（与 P 的最后一个字符对齐）考虑我们能得到什么信息：

观察 1

如果我们知道 $T[i]$ 这个字符不在 P 中，我们就不用考虑从 T 的第 i 个、第 $i+1$ 个……第 $i+m-1$ 个字符起出现的情况，而可以直接将 P 向下滑动 i 个字符。

观察 2

更一般地，如果出现在 P 最末尾（也就是最右边）的那一个字符的位置是离末尾端差了 k 个字符，

那么就可以不用匹配，直接将 P 向后滑动 k 个字符：如果滑动距离少于 k ，那么仅就 $T[i]$ 这个字符就无法被匹配，当然模式字符串 P 也就不会被匹配。

因此除非 $T[i]$ 字符可以和 P 末尾的那个字符匹配，否则要跳过 k 个字符（相当于 P 向后滑动了 k 个字符）。并且我们可以得到一个计算 k 的函数：

k 不在 P 中是上最后一个字符为出现在最末尾的那一个出现的位置，即

注意：显然这个表只需计算到 $P[m]$ 的位置

现在假设 $T[i]$ 和 P 最后一个字符匹配到了，那我们就看看 $T[i-1]$ 和 P 的倒数第二个字符是否匹配：

如果是，就继续回退直到整个模式串 P 完成匹配（这时我们就在 T 上成功得到了一个 P 的匹配）；

或者，我们也可能在匹配完 P 的倒数第 k 个字符后，在倒数第 k 个字符上失配，这时我们就希望把 P 向后滑动到下一个可能会实现匹配的位置，当然我们希望滑动得越远越好。

观察 3(a)

在 **观察 2** 中提到，当匹配完 P 的倒数 k 个字符后，如果在倒数第 k 个字符失配，为了使得 P 中的失配字符与 T 上对应字符对齐，

需要把 P 向后滑动 k 个字符，也就是说我们应该把注意力看向之后的 k 个字符（也就是看向 P 滑动 k 之后，末段与 T 对齐的那个字符）。

而，

所以我们的注意力应该沿着 T 向后跳 k 个字符。

然而，我们有机会跳过更多的字符，请继续看下去。

观察 3(b)

如果我们知道 接下来的 个字符和 的最后 个字符匹配，假设这个子串为 ，

我们还知道在 失配字符 后面是与 相匹配的子串，而假如 对应失配字符前面存在 ， 我们可以将 向下滑动一段距离，

使得失配字符 在 上对应的字符前面出现的 （合理重现，plausible reoccurrence，以下也简称 pr）与 的 对齐。如果 上有多个，按照从右到左的后缀匹配顺序，取第一个（rightmost plausible reoccurrence，以下也简称 rpr）。

假设此时 向下滑动的 个字符（也即 末尾端的 与其最右边的合理重现的距离）， 这样我们的注意力应该沿着 向后滑动 个字符，这段距离我们称之为：

假定 为 在 上失配时的最右边合理重现的位置，（这里只给出简单定义，在下文的算法设计章节里会有更精确的讨论），那么显然 。

所以有：

为失配字符在上对应字符的位置

于是我们在失配时，可以把把 上的注意力往后跳过 个字符

过程

箭头指向失配字符：

没有出现 中，根据 **观察 1**， 直接向下移动 个字符，也就是 7 个字符：

根据 **观察 2**，我们需要将 向下移动 4 个字符使得短横线字符对齐：

现在char. 匹配了，把 上的指针左移一步继续匹配：

根据 **观察 3(a)**，失配，因为 不在 中，所以 向下移动 个字符，而 上指针向下移动 个字符：

这时 又一次匹配到了 的最后一个字符， 上的指针向左匹配，匹配到了，继续向左匹配，发现在字符 失配：

显然直观上看，此时根据 **观察 3(b)**，将 向下移动 个字符，使得后缀 对齐，这种滑动可以获得 指针最大的滑动距离，此时，即 上指针向下滑动 7 个字符。

而从形式化逻辑看，此时，， 这样从形式逻辑上支持了进行 **观察 3(b)** 的跳转：

现在我们发现了 上每一个字符都和 上对应的字符相等，我们在 上找到了一个 的匹配。而只花费了 14 次对 的引用，其中 7 次是完成一个成功的匹配所必需的比较次数（），另外 7 次让我们跳过了 22 个字符。

算法设计

最初的匹配算法

解释

现在看这样一个利用 和 进行字符串匹配的算法：

如果上面的算法，表明 不在 中；如果返回一个数字，表示 在 左起第一次出现的位置。

然后让我们更精细地描述下计算，所依靠的 函数。

根据前文定义，表示在失配时，子串在 最右边合理重现的位置。

也就是说需要找到一个最好的，使得，另外要考虑两种特殊情况：

1. 当时，相当于在 前面补充了一段虚拟的前缀，实际上也符合 跳转的原理。
2. 当时，如果，则这个 不能作为 的合理重现。原因是 本身是失配字符，所以 向下滑动 个字符后，在后缀匹配过程中仍然会在 处失配。

还要注意两个限制条件：

1. 。因为当时，有，在 上失配的字符也会在 上失配。
2. 考虑到，所以规定。

过程

由于理解 是实现 BoyerMoore 算法的核心，所以我们使用如下两个例子进行详细说明：

对于，为，在 之前的最右边合理重现只能是，也就是最右边合理重现位置为 -5，即；

对于，为，在 之前的最右边的合理重现是，所以；

对于，为，在 之前的最右边的合理重现是，所以；

对于，为，在 之前的最右边的合理重现是，所以；

对于，为，在 之前的最右边的合理重现是，所以；

对于，为，在 之前的最右边的合理重现是，所以；

对于，为，又因为，即 等于失配字符，所以 并不是符合条件的 的合理重现，所以在最右边的合理重现是，所以；

对于，为，同理又因为，所以 并不是符合条件的 的合理重现，在最右边的合理重现是，所以；

对于，根据 定义，，得到。

现在再看一下另一个例子：

对于，为，在 之前的最右边合理重现只能是，也就是最右边合理重现位置为 -8，即；

对于，为，在 之前的最右边合理重现只能是，；

对于，为，在 之前的最右边合理重现只能是，；

对于，为，在 之前的最右边合理重现只能是，；

对于，为，在 之前的最右边合理重现只能是，；

对于，为，在 之前的最右边合理重现只能是，；

对于，为，因为 并且有，所以在 之前的最右边的合理重现是，；

对于，为，虽然 但是因为，所以在 之前的最右边的合理重现是，；

对于，根据 定义，，得到。

对匹配算法的一个改进

最后，实践过程中考虑到搜索过程中估计有 80% 的时间用在了 观察 1 的跳转上，也就是 和 不匹配，然后跳跃整个 进行下一次匹配的过程。

于是，可以为此进行特别的优化：

我们定义一个：

为一个整数，需要满足

用 `delta2` 代替，得到改进后的匹配算法：

除非和末尾字符匹配，否则至多向下滑动此时表示上没有一个字符和末尾字符匹配

其中 `delta2` 起到多重作用，一是类似后面介绍的 Horspool 算法进行快速的坏字符跳转，二是辅助检测字符串搜索是否完成。

经过改进，比起原算法，在做 **观察 1** 跳转时不必每次进行 的多余计算，使得在通常字符集下搜索字符串的性能有了明显的提升。

delta2 构建细节

引入

在 1977 年 10 月的 *Communications of the ACM* 上，Boyer、Moor 的论文¹中只描述了 静态表，

构造 的具体实现的讨论出现在 1977 年 6 月 Knuth、Morris、Pratt 在 *SIAM Journal on Computing* 上正式联合发表的 KMP 算法的论文²。

朴素算法

在介绍 Knuth 的 构建算法之前，根据定义，我们会有一个适用于小规模问题的朴素算法：

1. 对于 $[0, \text{patlen})$ 区间的每一个位置 i ，根据 `subpat` 的长度确定其重现位置的区间，也就是 $[-\text{subpatlen}, i]$ ；
2. 可能的重现位置按照从右到左进行逐字符比较，寻找符合 要求的最右边 的重现位置；
3. 最后别忘了令 。

▼ 实现

```
1 use std::cmp::PartialEq;
2
3 pub fn build_delta_2_table_naive(p: &[impl PartialEq]) -> Vec<usize> {
4     let patlen = p.len();
5     let lastpos = patlen - 1;
6     let mut delta_2 = vec![];
7
8     for i in 0..patlen {
9         let subpatlen = (lastpos - i) as isize;
10
11         if subpatlen == 0 {
12             delta_2.push(0);
13             break;
14         }
15
16         for j in (-subpatlen..(i + 1) as isize).rev() {
17             // subpat 匹配
18             if (j..j + subpatlen)
19                 .zip(i + 1..patlen)
20                 .all(|(rpr_index, subpat_index)| {
21                     if rpr_index < 0 {
22                         return true;
23                     }
24
25                     if p[rpr_index as usize] == p[subpat_index] {
26                         return true;
27                     }
28
29                     false
30                 })
31                 && (j <= 0 || p[(j - 1) as usize] != p[i])
32             {
33                 delta_2.push((lastpos as isize - j) as usize);
34                 break;
35             }
36         }
37     }
38
39     delta_2
40 }
```

特别地，对 Rust 语言特性进行必要地解释，下不赘述：

- `usize` 和 `isize` 是和内存指针同字节数的无符号整数和有符号整数，在 32 位机上相当于 `u32` 和 `i32`，64 位机上相当于 `u64` 和 `i64`。

- 显然，该暴力算法的时间复杂度为 $O(n^2)$ 。

下面我们要介绍的是时间复杂度为 $O(n^2)$ ，但是需要额外 $O(1)$ 空间复杂度的高效算法。

虽然 1977 年 Knuth 提出了这个构建方法，然而他的原始版本的构建算法存在一个缺陷，实际上对于某些 产生不出符合定义的。

Rytter 在 1980 年 *SIAM Journal on Computing* 上发表的文章³对此提出了修正, 以下是 的构建算法:

首先考虑到 的定义比较复杂，我们按照 的重现位置进行分类，每一类进行单独处理，这是高效实现的关键思路。

按照重现位置由远到近，也就是偏移量由大到小，分成如下几类：

- 现在来讨论如何高效地计算这三种情况：

这是最简单的情况，只需一次遍历并且可以顺便将 `initial` 初始化。

我们观察什么时候会出现 的重现一部分在 左边，一部分是 的头部的情况呢？应该是 的某个后缀和 的某个前缀相等，

比如之前的例子：

的重现， 的后缀与 pat 前缀中，有相等的，是。

实际上对第二种和第三种情况的计算的关键都需要前缀函数的计算和和应用

那么只要 ϵ 取值使得 ϵ 包含这个相等的后缀, 那么就可以得到第二种情况的 ϵ 的重现, 对于例子, 我们只需要使得,

而当时，就是完全在头部的边际情况。

可以计算此时的：

设此时这对相等的前后缀长度为 k ，可知，那么在左边的部分长度是，

而，所以得到。

其后面可能会有多对相等的前缀和后缀，比如：

在 处有, 处有, 在 处有

Knuth 算法的缺陷是只考虑了最长的那一对的情况，但实际上我们要考虑所有 后缀与 前缀相等的情况，等同于计算 所有真后缀和真前缀相等的情况，并按照长度从大到小， 分区间计算不同的。

利用前缀函数和逆向运用计算前缀函数的状态转移方程：，以得到 所有相等的真前缀和真后缀长度。从 开始作为最长一对的长度，然后通过逆向运行状态转移方程，得到下一个次长相等的真前缀和真后缀的长度。

如此就完成了第二种情况的 的计算。

第三种情况

的重现恰好就在 S 中（不包括 S 的头部），也就是按照从右到左的顺序，在 S 中寻找 S 。

如果用 BM 算法解决，我们就得到了一个 BM 的递归实现的第三种情况，结束条件是

而且根据 S 的定义，找到的 S 的重现的下一个（也就是左边一个）字符和作为 S 后缀的 S 的下一个字符不能一样。

这就很好地启发了我们，可以使用类似于计算前缀函数的过程计算第三种情况，只不过是左右反过来的前缀函数：

- 两个指针分别指向子串的左端点和子串最长公共前后缀的「前缀」位置，从右向左移动，在发现指向的两个字符相等时继续移动，此时相当于「前缀」变大；
- 当两个字符不相等时，之前相等的部分就满足了 S 对重现的要求，并且回退指向「前缀」位置的指针直到构成新的字符相等或者出界。

同前缀函数一样，需要一个辅助数组，用于回退，可以使用之前计算第二种情况所生成的前缀数组的空间。

实现

► 上述实现

Galil 规则对多次匹配时最坏情况的改善

关于后缀匹配算法的多次匹配问题

之前的搜索算法只涉及到在 S 中寻找第一次 S 匹配的情况，而对与在 S 中寻找全部 S 的匹配的情况有很多不同的算法思路，这个问题的核心关注点是：

如何利用之前匹配成功的字符的信息，将最坏情况下的时间复杂度降为线性。

在原始的成功匹配后，简单的 S 的指针向后滑动 S 距离后重新开始后缀匹配，这会导致最坏情况下回到 S 的时间复杂度（按照惯例， S 为 n ， S 为 n ，下同）。

比如一个极端的例子： S ， S ， S 。

对此 Knuth 提出来的一个方法是用一个「数量有限」的状态的集合来记录 S 长度的字符，这种算法保证 S 上每一个字符最多比较一次，但代价是这个「数量有限」的状态可能规模并不小，对于一个字符彼此不相等的 S ，需要 S 个状态。

下面介绍的思路简单且不需要额外预处理开销的 Galil 算法⁴。

Galil 规则

假定一个 S ，它是某个子串 S 重复 n 次构成的字符串 S 的前缀，那么我们称 S 为 S 的一个周期。

比如， S ， S 是 S 的重复 S 的前缀，所以 S 的长度 S 就是这个 S 的周期长度，也即 S 满足 S 。

至少拥有一个长度为它自身的周期，我们规定最短的周期为 S ， S 。

在搜索过程中，假如我们的 S 成功地完成了一次匹配，那么依照周期的特点，实际上只需将 S 向后滑动 S 个字符，比较这 S 个字符是否对应相等就可以直接判断是否存在 S 的又一个匹配。

为计算这个最短周期的长度，我们假设已知 S 的相等的一对前缀 - 后缀，设它们的长度为 S ，那么有 S 。从而得到长度为 S 的周期，

当我们知道 S 最长的那一对相等的前缀 - 后缀，我们就得到了 S 最短的周期。

而最长相等的前后缀长度 S ，已经在我们在计算 S 的过程中，所以实际不需要额外的预处理时间和空间，就能将后缀匹配算法最坏情况的时间复杂度改善成线性。

► 结合上述优化的 BM 的搜索算法最终实现

最坏情况在实践中性能影响

从实践的角度上说，理论上的最坏情况并不容易影响性能表现，哪怕是很小的只有 4 的字符集的随机文本测试下这种最坏情况的影响也小到难以观察。

也因此如果没有很好地设计，使用 Galil 法则会拖累一点平均的性能表现，但对于一些极端特殊的 S 和 S 比如例子中的 S ， S ，Galil 规则的应用确实会使得性能表现提高数倍。

改进算法

Simplified Boyer–Moore 算法

BM 算法最复杂的地方就在于 表（也就是好后缀表）的构建，而实践中发现，在一般的字符集上的匹配性能主要依靠 表（也就是坏字符表），于是出现了仅仅使用 表的简化版 BM 算法，通常性能和原版差距很小。

Boyer–Moore–Horspool 算法

Horspool 算法同样是基于坏字符的规则，在与 尾部对齐的字符上应用 。效果类似于对原版匹配算法的改进，通常性能优于原版本。

▼ 实现

```
1 pub struct HorspoolPattern<'a> {
2     pat_bytes: &'a [u8],
3     bm_bc: [usize; 256],
4 }
5
6 impl<'a> HorspoolPattern<'a> {
7     // ...
8     pub fn find_all(&self, string: &str) -> Vec<usize> {
9         let mut result = vec![];
10        let string_bytes = string.as_bytes();
11        let stringlen = string_bytes.len();
12        let pat_last_pos = self.pat_bytes.len() - 1;
13        let mut string_index = pat_last_pos;
14
15        while string_index < stringlen {
16            if &string_bytes[string_index-pat_last_pos..string_index+1] == self.pat_bytes {
17                result.push(string_index-pat_last_pos);
18            }
19
20            string_index += self.bm_bc[string_bytes[string_index] as usize];
21        }
22
23        result
24    }
25 }
```

Boyer–Moore–Sunday 算法

Sunday 算法同样是利用坏字符规则，只不过相比 Horspool 它更进一步，直接关注 尾部对齐的那个字符的下一个字符。

实现它只需要稍微修改 表，相当于在 长度的 上进行构建。

Sunday 算法通常用作一般情况下实现最简单而且平均表现最好之一的实用算法，通常性能比 Horspool 和 BM 要好一点。

▼ 实现

```

1 pub struct SundayPattern<'a> {
2     pat_bytes: &'a [u8],
3     sunday_bc: [usize; 256],
4 }
5
6 impl<'a> SundayPattern<'a> {
7     // ...
8     fn build_sunday_bc(p: &'a [u8]) -> [usize; 256] {
9         let mut sunday_bc_table = [p.len() + 1; 256];
10
11         for i in 0..p.len() {
12             sunday_bc_table[p[i] as usize] = p.len() - i;
13         }
14
15         sunday_bc_table
16     }
17
18     pub fn find_all(&self, string: &str) -> Vec<usize> {
19         let mut result = vec![];
20         let string_bytes = string.as_bytes();
21         let pat_last_pos = self.pat_bytes.len() - 1;
22         let stringlen = string_bytes.len();
23         let mut string_index = pat_last_pos;
24
25         while string_index < stringlen {
26             if &string_bytes[string_index - pat_last_pos..string_index+1] == self.pat_bytes {
27                 result.push(string_index - pat_last_pos);
28             }
29
30             if string_index + 1 == stringlen {
31                 break;
32             }
33
34             string_index += self.sunday_bc[string_bytes[string_index + 1] as usize];
35         }
36
37         result
38     }
39 }

```

BMHBNFS 算法

该算法结合了 Horspool 和 Sunday，是 CPython 实现 `stringlib` 模块时用到的 `find` 的算法⁵，以下简称 B5S。

B5S 基本思路是：

1. 按照后缀匹配的思路，首先比较 位置对应的字符是否相等，如果相等就比较 对应位置的字符是否相等，如果仍然相等，那么就发现一个匹配；
2. 如果任何一个阶段发生不匹配，就进入跳转阶段；
3. 在跳转阶段，首先观察 位置的下一个字符是否在 中，如果不在，直接向右滑动，这是 Sunday 算法的最大利用；
如果这个字符在 中，对 处的字符利用 进行 Horspool 跳转。

而根据时间节省还是空间节省为第一目标，算法会有差别巨大的不同实现。

时间节省版本

▼ 实现


```

1 pub struct B5TimePattern<'a> {
2     pat_bytes: &'a [u8],
3     alphabet: [bool;256],
4     bm_bc: [usize;256],
5     k: usize
6 }
7
8 impl<'a> B5TimePattern<'a> {
9     pub fn new(pat: &'a str) -> Self {
10         assert_ne!(pat.len(), 0);
11
12         let pat_bytes = pat.as_bytes();
13         let (alphabet, bm_bc, k) = B5TimePattern::build(pat_bytes);
14
15         B5TimePattern { pat_bytes, alphabet, bm_bc, k }
16     }
17
18     fn build(p: &'a [u8]) -> ([bool;256], [usize;256], usize) {
19         let mut alphabet = [false;256];
20         let mut bm_bc = [p.len(); 256];
21         let lastpos = p.len() - 1;
22
23         for i in 0..lastpos {
24             alphabet[p[i] as usize] = true;
25             bm_bc[p[i] as usize] = lastpos - i;
26         }
27
28         alphabet[p[lastpos] as usize] = true;
29
30         (alphabet, bm_bc, compute_k(p))
31     }
32
33     pub fn find_all(&self, string: &str) -> Vec<usize> {
34         let mut result = vec![];
35         let string_bytes = string.as_bytes();
36         let pat_last_pos = self.pat_bytes.len() - 1;
37         let patlen = self.pat_bytes.len();
38         let stringlen = string_bytes.len();
39         let mut string_index = pat_last_pos;
40         let mut offset = pat_last_pos;
41         let offset0 = self.k - 1;
42
43         while string_index < stringlen {
44             if string_bytes[string_index] == self.pat_bytes[pat_last_pos] {
45                 if &string_bytes[string_index-offset..string_index] == &self.pat_bytes[pat_last_pos-offset..pat_last_pos] {
46                     result.push(string_index-pat_last_pos);
47
48                     offset = offset0;
49
50                     // Galil rule
51                     string_index += self.k;
52                     continue;
53                 }
54             }
55
56             if string_index + 1 == stringlen {
57                 break;
58             }
59
60             offset = pat_last_pos;
61
62             if !self.alphabet[string_bytes[string_index+1] as usize] {
63                 string_index += patlen + 1; // sunday
64             } else {
65                 string_index += self.bm_bc[string_bytes[string_index] as usize]; // horspool
66             }
67         }
68
69         result
70     }
71 }

```

该版本的 B5S 性能表现非常理想，在目前介绍的后缀匹配系列算法中是通常情况下是最快的。

空间节省版本

同样在 CPython `stringlib` 中实现，使用了两个整数近似取代了字符表和 `isalnum` 的作用，极大地节省了空间：

1. 用一个简单的 Bloom 过滤器取代字符表（alphabet）

▼ 实现

```
1 pub struct BytesBloomFilter {
2     mask: u64,
3 }
4
5 impl BytesBloomFilter {
6     pub fn new() -> Self {
7         SimpleBloomFilter {
8             mask: 0,
9         }
10    }
11
12    fn insert(&mut self, byte: &u8) {
13        (self.mask) |= 1u64 << (byte & 63);
14    }
15
16    fn contains(&self, char: &u8) -> bool {
17        (self.mask & (1u64 << (byte & 63))) != 0
18    }
19 }
```

Bloom 过滤器设计通过牺牲准确率（实际还有运行时间）来极大地节省存储空间的 `Set` 类型的数据结构，它的特点是会将集中不存在的项误判为存在（False Positives，简称 FP），但不会把集中存在的项判断为不存在（False Negatives，简称 FN），因此使用它可能会因为 FP 而没有得到最大的字符跳转，但不会因为 FN 而跳过本应匹配的字符。

理论上分析，上述「Bloom 过滤器」的实现在 长度在 50 个 Bytes 时，FP 概率约为 0.5，而 长度在 10 个 Bytes 时，FP 概率约为 0.15。

虽然这不是一个标准的 Bloom 过滤器，首先它实际上没有使用一个真正的哈希函数，实际上它只是一个字符映射，将 0-255 的字节映射为它的前六位构成的数。

但考虑到我们在内存上的进行字符搜索，这种简化就非常重要，即使用目前已知最快的非加密哈希算法 [xxHash](#)，计算所需要的时间仍比它高一个数量级。

另外当 `pat` 在 30 字节以下时，为了达到最佳的 FP 概率，需要不止一个哈希函数。但这么做意义不大，因为用装有两个 `u128` 数字的数组就已经可以构建字符表的全字符集。

2. 使用 `isalnum` 代替整个

观察，最常使用处就是后缀匹配时第一个字符就不匹配是最常见的不匹配的情况，于是令 `skip = delta1(pat[patlastpos])`，

在第一阶段不匹配时，直接向下滑动 `skip` 个字符；但当第二阶段不匹配时，因为缺乏整个 `pat` 的信息，只能向下滑动一个字符。

▼ 实现

```
1 pub struct B5SSpacePattern<'a> {
2     pat_bytes: &'a [u8],
3     alphabet: BytesBloomFilter,
4     skip: usize,
5 }
6
7 impl<'a> B5SSpacePattern<'a> {
8     pub fn new(pat: &'a str) -> Self {
9         assert_ne!(pat.len(), 0);
10    }
11
12    let pat_bytes = pat.as_bytes();
13    let (alphabet, skip) = B5SSpacePattern::build(pat_bytes);
14
15    B5SSpacePattern { pat_bytes, alphabet, skip }
16 }
17
18 fn build(p: &'a [u8]) -> (BytesBloomFilter, usize) {
19     let mut alphabet = BytesBloomFilter::new();
20     let lastpos = p.len() - 1;
```

```

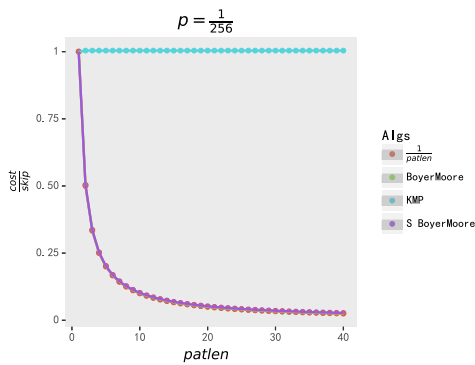
20     let mut skip = p.len();
21
22     for i in 0..p.len()-1 {
23         alphabet.insert(&p[i]);
24
25         if p[i] == p[lastpos] {
26             skip = lastpos - i;
27         }
28     }
29
30     alphabet.insert(&p[lastpos]);
31
32     (alphabet, skip)
33 }
34
35 pub fn find_all(&self, string: &'a str) -> Vec<usize> {
36     let mut result = vec![];
37     let string_bytes = string.as_bytes();
38     let pat_last_pos = self.pat_bytes.len() - 1;
39     let patlen = self.pat_bytes.len();
40     let stringlen = string_bytes.len();
41     let mut string_index = pat_last_pos;
42
43     while string_index < stringlen {
44         if string_bytes[string_index] == self.pat_bytes[pat_last_pos] {
45             if &string_bytes[string_index-pat_last_pos..string_index] == &self.pat_bytes[..patlen-1] {
46                 result.push(string_index-pat_last_pos);
47             }
48
49             if string_index + 1 == stringlen {
50                 break;
51             }
52
53             if !self.alphabet.contains(&string_bytes[string_index+1]) {
54                 string_index += patlen + 1; // sunday
55             } else {
56                 string_index += self.skip; // horspool
57             }
58         } else {
59             if string_index + 1 == stringlen {
60                 break;
61             }
62
63             if !self.alphabet.contains(&string_bytes[string_index+1]) {
64                 string_index += patlen + 1; // sunday
65             } else {
66                 string_index += 1;
67             }
68         }
69     }
70
71     result
72 }
73
74 }

```

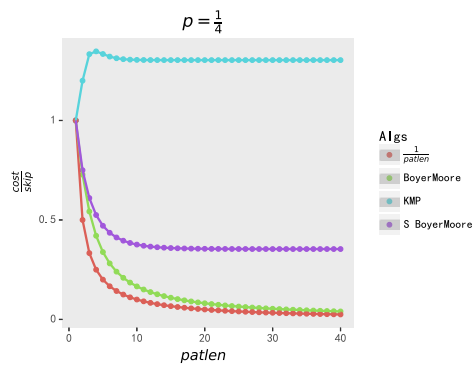
这个版本的算法相较于前面的后缀匹配算法不够快，但差距不大，性能仍然优于 KMP，得益于它至多两个 `u64` 的整数的优秀空间复杂度。

理论分析

以下是一般字符集下各算法的表现，纵坐标类似于执行开销（`cost` 指匹配成功 `m` 个字符后失配时的代价，`skip` 指发生失配时向下滑动 `k` 个字符的概率），越小性能越好。横坐标为模式字符串 `pat` 的长度：



在较小字符集（DNA {A, C, T, G} 碱基对序列）中的表现：



综上，在较大的字符集，比如日常搜索的过程中，BoyerMoore 系列算法的优越表现，其中主要依赖 表实现字符跳转；

另一方面，在较小的字符集里， 的作用下降，而 的作用得到了体现。

如果有一定富裕空间的情况下，完整的空间复杂度为 的 BoyerMoore 算法更加通用，综合表现最优。

参考资料与注释

1. [1977 年 Boyer–Moore 算法论文 ↩](#)
2. [1977 年 KMP 算法论文 ↩](#)
3. [1980 年 Rytter 纠正 Knuth 的论文 ↩](#)
4. [1979 年介绍 Galil 算法的论文 ↩](#)
5. [B5S 算法的介绍 ↩](#)

本页面最近更新：2024/11/14 16:17:47，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[minghu6](#), [c0nstexpr](#), [CCXXI](#), [Chrogeek](#), [Early0v0](#), [Enter-tainer](#), [githuu5y5u](#), [HeRaNO](#), [iamtwz](#), [ksyx](#), [megakite](#), [Tiphereth-A](#), [wineee](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用